

Azure Deployment Plan

Conversational UI — Generative BI Report Hub (React frontend + Node/Express backend + Google BigQuery)

Prepared 11 June 2026 • Branch: vk4 • Audience: deployment / engineering

Summary. The app is a static React frontend plus a lightweight Node/Express backend that proxies to Claude/Gemini and queries Google BigQuery. It is **I/O-bound, not CPU-heavy**. Recommended VM: **Azure A2 v2 (2 vCPU, 4 GB RAM)**, or — better — **Azure App Service** (no VM to manage). The database (**BigQuery**) **stays on Google** and is reached over its API; **no DB migration is required**. The single most important code change is removing a **hardcoded `http://localhost:3001`** backend URL in the frontend.

1. What we are deploying

Piece	Stack	Build output	Runtime need
Frontend	React + Vite + MUI/Radix	<code>vite build</code> → <code>/dist</code> (static HTML/JS/CSS)	Any static host / CDN. No server CPU.
Backend	Node 18+ / Express / TypeScript	<code>tsc</code> → <code>backend/dist</code> , run <code>node dist/index.js</code>	Always-on Node process, port 3001 (configurable).
Database	Google BigQuery (serverless)	—	API access only. Lives on Google. No migration.
AI	Anthropic Claude + Google Gemini	—	External APIs. Need keys.

2. Hosting choice

Option	Best for	Effort	Notes
Azure App Service (recommended)	Backend Node app	Low	No OS to patch. Auto HTTPS, easy scale, env vars in portal. Must enable WebSockets / disable response buffering for SSE streaming.
Azure Static Web Apps (recommended)	Frontend	Low	Free/cheap, global CDN, built-in SPA fallback to <code>index.html</code> .
Azure VM A2 v2	Both on one box	Medium	You manage OS, Node, Nginx, HTTPS, restarts. Pick only if you need full control.

Recommendation: Frontend → Azure Static Web Apps. Backend → Azure App Service (Linux, Node 18/20). Simpler and cheaper than a raw VM, and it handles HTTPS + scaling for you. If policy mandates a VM, use **A2 v2** (see sizing in §6).

3. Code changes required (before deploy)

MUST 3.1 — Remove hardcoded backend URL in frontend

The streaming call points at localhost. In production this fails (browser tries to reach the user's own machine).

File: `src/app/pages/Conversational_new.tsx` — line ~2578 (and error text line ~2658).

```
// BEFORE – hardcoded, breaks in production
const res = await fetch("http://localhost:3001/api/conversational/stream", {

// AFTER – use env var, same pattern already used in BigQueryDashboard.tsx
const API_BASE = import.meta.env.VITE_API_URL || "http://localhost:3001";
const res = await fetch(`${API_BASE}/api/conversational/stream`, {
```

Then set `VITE_API_URL=https://<your-backend>.azurewebsites.net` at frontend build time. Note: `src/app/components/BigQueryDashboard.tsx` already does this correctly — copy that pattern.

SHOULD 3.2 — Lock down CORS on backend

Backend currently allows `Access-Control-Allow-Origin: *` (any site). For production, restrict to your frontend domain.

File: `backend/src/index.ts` — line ~16.

```
// Replace the wildcard "*" with your real frontend origin
const ALLOWED = process.env.FRONTEND_ORIGIN || "*";
res.header("Access-Control-Allow-Origin", ALLOWED);
// also update the same header inside the SSE res.writeHead(...) block (~line 33)
```

GOOD 3.3 — Confirm port + start command

Backend already reads `process.env.PORT` (good — Azure App Service injects its own PORT). Ensure start command is `npm run build && npm start` (runs `tsc` then `node dist/index.js`).

4. Environment variables / secrets

Currently in a local `.env` file. **Do NOT commit secrets**. Set these in Azure App Service → Configuration → Application settings (and as build-time vars for the frontend host).

Backend (Azure App Service settings)

Variable	Purpose	Source
<code>ANTHROPIC_API_KEY</code>	Claude AI calls	Anthropic console
<code>PORT</code>	Listen port	Auto-set by Azure (leave unset)
<code>BQ_PROJECT_ID</code>	BigQuery project (<code>data-practice-472314</code>)	GCP console
<code>BQ_DATASET</code>	Dataset (<code>report_hub_demo</code>)	GCP console
<code>BQ_CLIENT_EMAIL</code>	Service-account email	GCP service-account JSON
<code>BQ_PRIVATE_KEY</code>	Service-account private key	GCP service-account JSON

Gemini key if used (`@google/genai`) — add the relevant `GOOGLE_API_KEY` / `GEMINI_API_KEY` as the code expects.

Frontend (build-time)

Variable	Value
<code>VITE_API_URL</code>	<code>https://<backend>.azurewebsites.net</code>

BigQuery access from Azure. Because the backend now runs *outside* Google, it can no longer rely on implicit GCP credentials. It must authenticate with a **service-account key** (`BQ_CLIENT_EMAIL` + `BQ_PRIVATE_KEY`) — the code already supports this path in `backend/src/lib/bigqueryClient.ts`. Create/download a service-account JSON in GCP with *BigQuery Data Viewer* + *Job User* roles, then paste the two values into Azure settings. When pasting the private key, keep the `\n` escaping (the code already converts `\\n` back to real newlines).

5. The database question (answered)

Stays on Google. No migration needed. BigQuery is serverless and API-only by design — even when the backend ran on Google it talked to BigQuery over the network. Moving the backend to Azure changes nothing except that you now authenticate with a service-account key (§4).

- **Adding more datasets later** = just more queries. No effect on the Azure VM/app size — BigQuery does the heavy lifting on Google's side.
- **Watch cost:** BigQuery bills per data scanned, plus small network egress for results leaving Google. Not the VM.
- **Only migrate if** you later want everything in one cloud for latency/billing simplicity — optional, not required.

6. VM sizing (if using a VM)

Instance	vCPU / RAM	Temp storage	~ Price	Verdict
A2 v2	2 / 4 GB	20 GB	~\$0.090/hr (~\$66/mo)	Pick this — newer, more RAM, cheaper
A2	2 / 3.5 GB	135 GB	~\$0.120/hr (~\$88/mo)	Old gen, less RAM, costs more — skip

Temp storage doesn't matter here. It is ephemeral scratch space (wiped on reboot), not where your app or data lives. Your app stores almost nothing locally (static files + Node proxy; data is in BigQuery), so 20 GB is plenty.

Expected usage on A2 v2 today

Component	RAM	CPU
Linux OS	~0.4–0.6 GB	idle
Node backend (I/O-bound proxy)	~0.15–0.4 GB	~0% idle, brief spikes/request
Static frontend serving	~0.05 GB	trivial
Typical total	~1–1.5 GB of 4 GB (≈30–40%)	<20%

Headroom: plenty for current use and growth in *data*. The thing that eventually forces a bigger box is **many concurrent live users** (Node is single-threaded; LLM streaming uses CPU+RAM). When RAM stays >80% or CPU is pegged in normal use, resize to **B4ms / D2 v2** (4 vCPU, 8–16 GB) — a few minutes of downtime.

7. Step-by-step deployment

1. **Make code changes** §3 (hardcoded URL → env var; lock CORS).
2. **Create GCP service account** with BigQuery Data Viewer + Job User; download JSON.
3. **Backend** → **Azure App Service** (Linux, Node 20). Set env vars §4. Deploy via GitHub Actions / Azure CLI / zip.
 - Enable **WebSockets** and ensure response buffering is off so the `/stream` SSE endpoint works.
 - Start command: `npm run build && npm start` in `/backend`.
4. **Frontend** → **Azure Static Web Apps**. Set `VITE_API_URL` = backend URL. Build with `vite build`, serve `/dist`. SPA fallback to `index.html` (Static Web Apps does this automatically; the existing `vercel.json` rewrite is the equivalent).
5. **Set CORS:** `FRONTEND_ORIGIN` on backend = the Static Web App URL.
6. **Smoke test:** open the site, send a query, confirm streaming reply + a BigQuery-backed report render.
7. **Monitor:** Azure metrics (CPU/RAM) + BigQuery cost dashboard. Resize VM/plan only if usage stays high.

8. Anything missing? — Checklist of gaps to confirm

Item	Status / action
Hardcoded <code>localhost:3001</code> in <code>Conversational_new.tsx</code>	MUST FIX — §3.1
Backend CORS wildcard <code>*</code>	TIGHTEN — §3.2
BigQuery service-account key for off-GCP auth	CREATE — §4
SSE streaming through Azure (buffering/WebSockets)	VERIFY — §7 step 3
Secrets out of <code>.env</code> → Azure settings; confirm <code>.env</code> is git-ignored	CONFIRM

Gemini / Google GenAI key (<code>@google/genai</code>) — is it used in prod path?	CONFIRM which AI keys are actually required
HTTPS / custom domain	Handled by App Service + Static Web Apps; add custom domain if needed
Node version pin (engines field) + Node 18+ on host	RECOMMEND pin in <code>package.json</code>
Logging / health endpoint	<code>GET /</code> health check exists — wire Azure health probe to it
No automated tests / CI gate before deploy	OPTIONAL add a build check in CI

Bottom line. This is a clean, low-risk deployment. One required code fix (the hardcoded URL), one new credential (BigQuery service account), and correct env-var wiring. The database stays on Google untouched. Use Azure App Service + Static Web Apps for the easiest path, or an A2 v2 VM if you need a single box.